

Reference Notes

Week 13: Functions Calling Functions

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How can we break down complex problems into manageable pieces using functions? This week, we'll discover how functions can call other functions, creating powerful chains of operations that make our programs organized, readable, and easier to debug. We'll bring together everything we've learned to build complete, well-structured programs.

Prerequisites

You should be comfortable with:

- Creating functions with parameters and return values
- Understanding function scope and local variables
- Drawing environment diagrams for function calls
- Using all data structures: lists, strings, and dictionaries

1.2 What You Already Know

You've learned to create functions that take parameters and return values. You understand that variables inside functions are local and don't affect the outside world (unless you return them). You can trace through function calls using environment diagrams. Now we'll take the next step: building programs where functions work together.

1.3 What You'll Be Able to Do

By the end of this week, you'll be able to design programs with multiple functions that call each other, trace execution through chains of function calls, and build complete programs that solve real problems. You'll understand how professional programmers organize their code and why breaking problems into small functions makes programming easier and more enjoyable.

2 VIDEO 1: Understanding Functions Calling Functions

2.1 Why Functions Call Other Functions

In real programs, complex tasks are broken into smaller subtasks. Each function handles one specific job, and functions call other functions to get help with parts of their work. This is like how a chef might ask an assistant to chop vegetables while they prepare the sauce - each person has a specific role, and they work together to create the final dish.

2.2 Your First Function Chain

Let's see a simple example where one function calls another. We'll create a program that calculates the area of a rectangle, but uses a helper function:

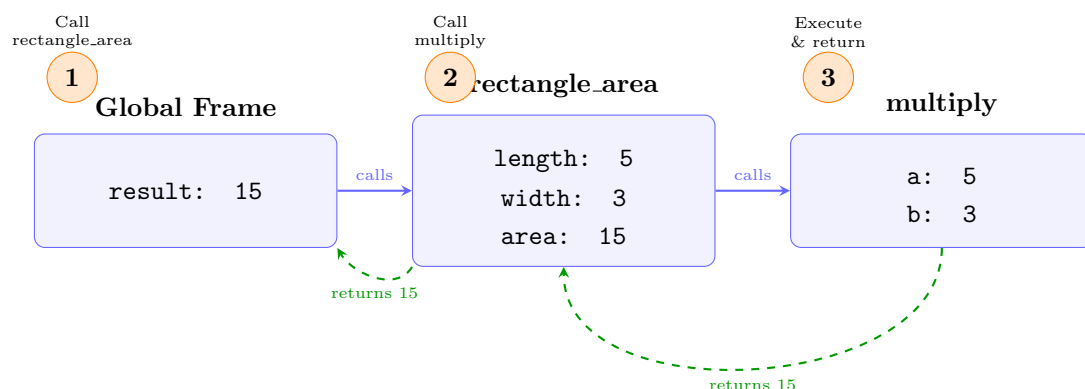
```

1 def multiply(a, b):
2     """Returns the product of two numbers."""
3     return a * b
4
5 def rectangle_area(length, width):
6     """Calculates rectangle area using multiply."""
7     area = multiply(length, width)
8     return area
9
10 # Using our functions
11 result = rectangle_area(5, 3)
12 print(result) # Output: 15
    
```

Notice how `rectangle_area` doesn't do the multiplication itself - it delegates that work to the `multiply` function. This might seem unnecessary for simple multiplication, but the pattern becomes powerful for complex tasks.

2.3 Tracing Function Call Chains

Understanding how Python executes function chains is crucial. Let's visualize what happens when we call `rectangle_area(5, 3)`:



The diagram shows how Python creates a new frame for each function call. When `multiply` finishes, its return value goes back to `rectangle_area`, which then returns to the global frame.

2.4 Building Longer Chains

Functions can call functions that call other functions, creating longer chains. Here's a more practical example that processes student grades:

```

1 def calculate_average(numbers):
2     """Returns the average of a list of numbers."""
3     total = sum(numbers)
4     return total / len(numbers)
5
6 def get_letter_grade(score):
7     """Converts a numeric score to a letter grade."""
8     if score >= 90: return 'A'
9     if score >= 80: return 'B'
10    if score >= 70: return 'C'
11    return 'F'
12
13 def process_student(grades):
14     """Gets average and letter grade for a student."""
15     avg = calculate_average(grades)
16     letter = get_letter_grade(avg)
17     return avg, letter

```

Each function has one clear job: calculate an average, convert to a letter, or coordinate the process. This makes each piece easy to understand and test.

Tip

When designing functions, ask yourself: "Does this function do ONE thing?" If it does multiple things, consider breaking it into smaller functions.

Quick Check

If we call `process_student([85, 90, 78])`, which functions get called and in what order?

3 VIDEO 2: Advanced Scope and Variable Lifetime

3.1 Scope in Nested Function Calls

When functions call other functions, each function has its own local scope. Variables in one function cannot see or affect variables in another function, even if they have the same name. This isolation is a feature, not a bug - it lets us write functions without worrying about name conflicts.

3.2 Same Names, Different Variables

Let's explore what happens when different functions use the same variable names:

```

1 def outer_function():
2     x = 10
3     result = inner_function()
4     print(f"outer x: {x}") # Still 10!
5     return result
6
7 def inner_function():
8     x = 99 # Different x, local to inner_function
9     return x * 2

```

```

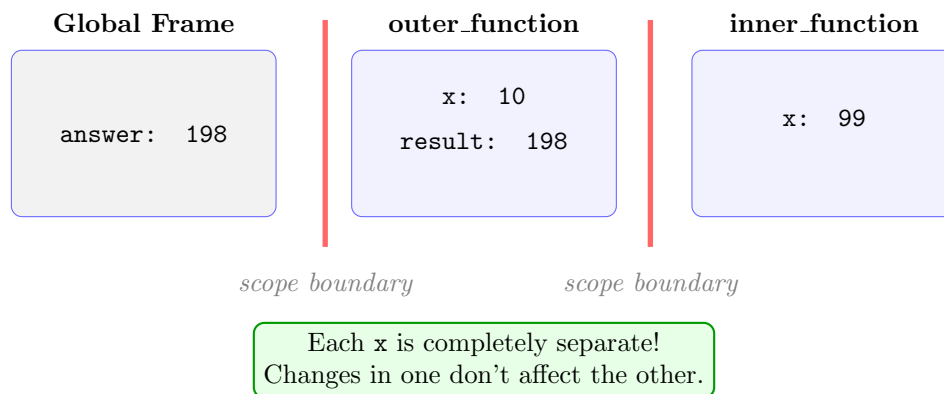
10
11 answer = outer_function()
12 print(f"answer: {answer}") # 198

```

Even though both functions have a variable called `x`, they are completely separate. The `x` in `inner_function` doesn't affect the `x` in `outer_function`.

3.3 Visualizing Scope Isolation

This environment diagram shows how each function gets its own isolated workspace:



The red walls represent scope boundaries. Variables cannot cross these boundaries - they exist only within their own function's frame.

3.4 Passing Data Between Functions

Since functions can't see each other's variables directly, we pass data through parameters and return values. This is the proper way for functions to communicate:

```

1 def double(n):
2     return n * 2
3
4 def add_ten(n):
5     return n + 10
6
7 def process(value):
8     # Pass data through parameters and returns
9     step1 = double(value)
10    step2 = add_ten(step1)
11    return step2
12
13 result = process(5) # double(5)=10, add_ten(10)=20
14 print(result) # 20

```

Data flows through our program like water through pipes - in through parameters, out through return values. This makes it easy to trace where values come from.

⚠ Common Error

Common Mistake: Expecting a variable from one function to be accessible in another. Each function has its own local variables. Use parameters to pass data in and return values to pass data out.

Quick Check

What will this print?

```
1 def func_a():
2     num = 5
3     return func_b()
4
5 def func_b():
6     return num * 2 # Will this work?
```

4 VIDEO 3: Building Complete Programs

4.1 Program Design Philosophy

Professional programmers don't write programs top-to-bottom in one go. They break problems into pieces, write a function for each piece, and then connect them. This approach is called *modular programming*, and it makes complex programs manageable.

4.2 A Complete Example: Quiz Game

Let's build a complete quiz game that demonstrates good program structure. We'll use multiple functions that each handle one specific task:

```
1 def create_questions():
2     """Returns a list of quiz questions and answers."""
3     return [
4         {"q": "Capital of Pakistan?", "a": "islamabad"},
5         {"q": "2 + 2 = ?", "a": "4"},
6         {"q": "Largest planet?", "a": "jupiter"}
7     ]
8
9 def ask_question(question_dict):
10    """Asks one question, returns True if correct."""
11    print(question_dict["q"])
12    answer = input("Your answer: ").lower().strip()
13    return answer == question_dict["a"]
```

Each function does exactly one thing: `create_questions` prepares the data, and `ask_question` handles a single question interaction.

4.3 Connecting the Pieces

Now we add functions to run the game and display results:

```
1 def run_quiz(questions):
2     """Runs the quiz, returns the score."""
3     score = 0
4     for q in questions:
5         if ask_question(q):
6             print("Correct!\n")
7             score += 1
8         else:
9             print(f"Wrong! Answer: {q['a']}\n")
10    return score
```

```

11
12 def show_results(score, total):
13     """Displays the final results."""
14     percent = (score / total) * 100
15     print(f"Score: {score}/{total} ({percent:.0f}%)")

```

Notice how `run_quiz` calls `ask_question` for each question. The functions work together but remain independent.

4.4 The Main Function Pattern

Professional programs use a `main` function to coordinate everything:

```

1 def main():
2     """Main function - coordinates the program."""
3     print("Welcome to the Quiz!\n")
4     questions = create_questions()
5     score = run_quiz(questions)
6     show_results(score, len(questions))
7     print("Thanks for playing!")
8
9 # Run the program
10 main()

```

The `main` function reads like a high-level description of what the program does: create questions, run the quiz, show results. Each step calls another function that handles the details.

💡 Rule

The main function should read like a summary of your program. Each line should describe WHAT happens, while the helper functions handle HOW it happens.

👉 Quick Check

Looking at the quiz program, in what order do the functions get called when we run `main()`?

5 VIDEO 4: Debugging Multi-Function Programs

5.1 Tracing Through Complex Programs

When programs have multiple functions, bugs can be harder to find. The key is systematic tracing - following the data through each function call to see where things go wrong.

5.2 Using Print Statements Strategically

Add print statements at function boundaries to see data flowing through your program:

```

1 def calculate_total(prices):
2     print(f"DEBUG: calculate_total received {prices}")
3     total = sum(prices)
4     print(f"DEBUG: calculate_total returning {total}")
5     return total
6
7 def apply_discount(amount, percent):
8     print(f"DEBUG: apply_discount({amount}, {percent})")

```

```

9     discount = amount * (percent / 100)
10    final = amount - discount
11    print(f"DEBUG: apply_discount returning {final}")
12    return final

```

These debug prints show you exactly what each function receives and returns. When something goes wrong, you can see exactly where the problem occurs.

5.3 Common Multi-Function Bugs

Here are patterns that often cause problems in programs with multiple functions:

Bug: Forgetting to return a value
Function returns `None` unexpectedly

Fix: Always check that functions
with results have `return` statements

Bug: Not storing the return value
`calculate(5)` instead of `x = calculate(5)`

Fix: Assign return values to variables
when you need to use them later

Bug: Wrong argument order
Parameters mixed up in the call

Fix: Use named arguments when unsure:
`func(width=5, height=3)`

5.4 Testing Functions Individually

Before connecting functions together, test each one separately:

```

1  # Test calculate_average alone
2  print(calculate_average([10, 20, 30]))  # Should be 20.0
3
4  # Test get_letter_grade alone
5  print(get_letter_grade(85))  # Should be 'B'
6  print(get_letter_grade(92))  # Should be 'A'
7
8  # Only after both work, test together
9  print(process_student([85, 90, 78]))

```

Testing functions individually makes it much easier to find bugs. If `process_student` gives wrong results but both helper functions work correctly, the bug must be in how `process_student` uses them.

Tip

When debugging, isolate the problem. Test each function separately, then test them together. The bug is where the behavior differs from what you expect.

Quick Check

A function should return a list but returns `None`. What's the most likely cause?

6 VIDEO 5: Summary

6.1 What We Learned This Week

This week, we discovered how to build programs where functions work together like a well-coordinated team. We learned that functions can call other functions, creating chains of operations that break complex problems into manageable pieces. We explored how scope keeps each function's variables isolated, and why we pass data through parameters and return values.

Most importantly, we learned the professional approach to program design: write small functions that each do one thing well, coordinate them with a main function, and test each piece before connecting them together.

★ Landmark Moment

Congratulations! You've completed Module 2! You now have all the tools to write real, well-organized programs: data structures (strings, lists, dictionaries), control flow (loops with break/continue), and functions that work together. You're thinking like a programmer!

7 Quick Reference

Quick Reference

Week 13 Essential Patterns: Functions Calling Functions:

- Pattern: Helper functions do small tasks, main function coordinates
- Common use: Breaking complex problems into manageable pieces
- Common mistake: Trying to access variables from other functions

Scope in Nested Calls:

- Pattern: Each function has its own local variables
- Memory: Variables exist only while their function runs
- Debug tip: Add print statements at function entry and exit

The Main Function Pattern:

- Pattern: `def main():` coordinates the program, then `main()`
- Common use: All professional programs use this structure
- Debug tip: Test helper functions before connecting them

7.1 Reflection Questions

1. What was the most challenging part about understanding how functions call other functions?
2. Can you explain to a friend why each function has its own separate variables?
3. When debugging a multi-function program, what's your strategy for finding where the bug is?
4. How has breaking problems into small functions changed how you think about programming?

5. Looking back at Module 2, what concept are you most proud of mastering?

Congratulations on completing Module 2! In Module 3, you'll explore object-oriented programming and build even more sophisticated programs!